```
        {
            q. UseMicrosoftDependencyInjectionJobFactory();

            var jobKey = new JobKey("Update exchange rates");   ❶
            q.AddJob<UpdateExchangeRatesJob>(opts =>            ❷
                opts.WithIdentity(jobKey));                     ❷

            q.AddTrigger(opts => opts                           ❸
                .ForJob(jobKey)                                 ❸
                .WithIdentity(jobKey.Name + " trigger")         ❹
                .StartNow()                                     ❺
                .WithSimpleSchedule(x => x                      ❻
                    .WithInterval(TimeSpan.FromMinutes(5))      ❻
                    .RepeatForever())
            );
        });

        services.AddQuartzHostedService(
            q => q.WaitForJobsToComplete = true);
    })
    .Build();
host.Run();
```

❶ Creates a unique key for the job, used to associate it with a trigger

❷ Adds the IJob to the DI container and associates it with the job key

❸ Registers a trigger for the IJob via the job key

❹ Provides a unique name for the trigger for use in logging and in clustered scenarios

❺ Fires the trigger as soon as the Quartz.NET scheduler runs on app startup

❻ Fires the trigger every 5 minutes until the app shuts down

Simple triggers like the schedule defined here are common, but you can also achieve more complex configurations using other schedules. The following configuration would set a trigger to fire every week on a Friday at 5:30 p.m.:

```
q.AddTrigger(opts => opts
    .ForJob(jobKey)
    .WithIdentity("Update exchange rates trigger")
    .WithSchedule(CronScheduleBuilder
        .WeeklyOnDayAndHourAndMinute(DayOfWeek.Friday, 17, 30)));
```